Asynchronous Programming with asyncio

8-Step Exercise Workbook - Solutions

ACTIVITY 1 - SOLUTION

Exercise 1: Simple Async Greeting

How The Solution Works

async_greet is declared with async def, which makes it a coroutine rather than an ordinary function. Calling await asyncio.sleep(0.3) inside it pauses only that coroutine and hands control back to the event loop for the duration of the sleep, instead of blocking the whole program. main awaits async_greet directly, so it receives the returned greeting string once the coroutine finishes, and asyncio.run(main()) creates the event loop, runs main to completion, and closes the loop again.

Key Points

- async def creates a coroutine function; calling it returns a coroutine object, not a result.
- await asyncio.sleep(0.3) yields control without blocking the thread.
- asyncio.run() is the standard entry point for a top-level coroutine.

Solution Code

```
"""
Solution 1: Defining and Running a Coroutine
Task: Declare a simple async greeting coroutine and run it using the asyncio loop.
"""
import asyncio


async def async_greet(name):
    print(f"Preparing greeting for {name}...")
    await asyncio.sleep(0.3)
    return f"Welcome, {name} to Async Programming!"


async def main():
    result = await async_greet("Delegate")
    print(f"Result: {result}")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Preparing greeting for Delegate...
Result: Welcome, Delegate to Async Programming!
```

ACTIVITY 2 - SOLUTION

Exercise 2: Concurrent Task Scheduling

How The Solution Works

Each call to asyncio.create_task() wraps a coroutine in a Task and schedules it on the event loop immediately, so all three fetch_api_endpoint calls start running as soon as they are created rather than waiting to be awaited. Because the tasks are already progressing in the background, awaiting them afterwards simply retrieves each result once it is ready. The slowest call, the 0.5 second /users request, sets the overall elapsed time, since the other two complete while it is still running.

Key Points

- create_task() starts a coroutine running now, not when it is later awaited.
- Awaiting a Task after it has already started only waits for the remaining time.
- Total elapsed time is bounded by the slowest task, not the sum of all delays.

Solution Code

```
"""
Solution 2: Concurrent Task Scheduling
Task: Wrap multiple instances of a slow coroutine in background Tasks to execute in parallel.
"""
import asyncio
import time


async def fetch_api_endpoint(endpoint, delay):
    print(f"Polling {endpoint}...")
    await asyncio.sleep(delay)
    print(f"Polled {endpoint} successfully.")
    return f"{endpoint}_data"


async def main():
    start = time.perf_counter()

    task1 = asyncio.create_task(fetch_api_endpoint("/users", 0.5))
    task2 = asyncio.create_task(fetch_api_endpoint("/products", 0.2))
    task3 = asyncio.create_task(fetch_api_endpoint("/metrics", 0.4))

    r1, r2, r3 = await task1, await task2, await task3

    elapsed = time.perf_counter() - start
    print(f"Results: {r1}, {r2}, {r3}")
    print(f"Completed in {elapsed:.2f} seconds.")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Polling /users...
Polling /products...
Polling /metrics...
Polled /products successfully.
Polled /metrics successfully.
Polled /users successfully.
Results: /users_data, /products_data, /metrics_data
Completed in 0.50 seconds.
```

ACTIVITY 3 - SOLUTION

Exercise 3: Bulk Gathering of Awaitables

How The Solution Works

A generator expression, fetch_product_inventory(pid) for pid in product_ids, produces one coroutine call per product id. Prefixing it with * inside asyncio.gather unpacks that generator into individual positional arguments, which is what gather expects. gather then runs every coroutine concurrently and returns a single list of results, ordered to match the sequence the coroutines were passed in, regardless of which one actually finishes first.

Key Points

- The unpacking operator (*) turns a generator or list into separate arguments for gather.
- gather scales naturally to any number of awaitables without writing one create_task call per item.
- Result order always matches input order, independent of completion order.

Solution Code

```
"""
Solution 3: Bulk Gathering of Awaitables
Task: Use asyncio.gather to aggregate dynamic counts of concurrent API queries.
"""
import asyncio


async def fetch_product_inventory(product_id):
    print(f"Fetching stock levels for ID: {product_id}...")
    await asyncio.sleep(0.3)
    return {product_id: 10 + (product_id % 5)}


async def main():
    product_ids = [101, 102, 103, 104, 105]

    results = await asyncio.gather(
        *(fetch_product_inventory(pid) for pid in product_ids)
    )

    print(f"Gathered inventory counts: {results}")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Fetching stock levels for ID: 101...
Fetching stock levels for ID: 102...
Fetching stock levels for ID: 103...
Fetching stock levels for ID: 104...
Fetching stock levels for ID: 105...
Gathered inventory counts: [{101: 11}, {102: 12}, {103: 13}, {104: 14}, {105: 10}]
```

ACTIVITY 4 - SOLUTION

Exercise 4: Bounded Web Requests with Timeouts

How The Solution Works

asyncio.wait_for wraps the slow download_heavy_archive() call with a 1.0 second ceiling. Internally, wait_for schedules the coroutine and races it against the timeout; if the coroutine has not finished when the timeout elapses, wait_for cancels it and raises asyncio.TimeoutError. The try-except block catches that specific exception and logs a fallback message instead of letting the exception propagate and crash the program.

Key Points

- wait_for(coro, timeout=...) enforces a maximum wait time on any awaitable.
- A timeout raises asyncio.TimeoutError rather than returning a partial result.
- Catching the specific exception keeps the except block from silently swallowing unrelated errors.

Solution Code

```
"""
Solution 4: Bounded Web Requests with Timeouts
Task: Apply a strict timeout boundary to a lagging query, handling errors gracefully.
"""
import asyncio


async def download_heavy_archive():
    print("Beginning massive archive download stream...")
    await asyncio.sleep(2.5)  # Simulate network drag
    return "ARCHIVE_BYTES"


async def main():
    try:
        data = await asyncio.wait_for(download_heavy_archive(), timeout=1.0)
        print(f"Success: {data}")
    except asyncio.TimeoutError:
        print("Network boundary exceeded! Download timed out. Falling back to local mirror.")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Beginning massive archive download stream...
Network boundary exceeded! Download timed out. Falling back to local mirror.
```

ACTIVITY 5 - SOLUTION

Exercise 5: State Synchronization with Locks

How The Solution Works

Ten record_page_view coroutines run concurrently through asyncio.gather, and each one reads shared_metrics, awaits a short sleep, then writes the incremented value back. Without protection, that read-sleep-write sequence could interleave, causing lost updates. Wrapping the critical section in async with metrics_lock ensures only one coroutine can be inside that section at a time; every other coroutine that reaches the lock suspends until it is released, so the final count is always correct.

Key Points

- A single asyncio.Lock instance is shared by every worker coroutine.
- async with lock acquires on entry and releases on exit, even if an exception occurs.
- The await inside the critical section is exactly where interleaving would otherwise happen.

Solution Code

```
"""
Solution 5: State Synchronization with Lock
Task: Safeguard a concurrent shared metrics dict from race conditions using a Lock.
"""
import asyncio

shared_metrics = {"page_views": 0}
metrics_lock = asyncio.Lock()


async def record_page_view(user_id):
    global shared_metrics
    print(f"User {user_id} browsing page...")
    async with metrics_lock:
        current = shared_metrics["page_views"]
        await asyncio.sleep(0.05)  # Intercept and force context switch
        shared_metrics["page_views"] = current + 1


async def main():
    await asyncio.gather(*(record_page_view(i) for i in range(10)))
    print(f"Expected page views: 10 | Actual recorded views: {shared_metrics['page_views']}")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
User 0 browsing page...
User 1 browsing page...
... (remaining worker logs)
Expected page views: 10 | Actual recorded views: 10
```

ACTIVITY 6 - SOLUTION

Exercise 6: Throttling Concurrency with Semaphores

How The Solution Works

api_semaphore is created with a capacity of 2, meaning at most two coroutines can hold it at once. Each call_restricted_api coroutine acquires the semaphore with async with api_semaphore before making its simulated request. When five calls are launched together through gather, the first two proceed immediately and the remaining three suspend until a slot is released. This staggers the five 0.4 second calls into three waves, producing a total runtime of roughly 1.2 seconds instead of 0.4 seconds.

Key Points

- asyncio.Semaphore(n) allows at most n concurrent holders.
- async with semaphore acquires a slot and releases it automatically afterwards.
- Throttling trades some concurrency for controlled load on a limited resource.

Solution Code

```
"""
Solution 6: Throttling Concurrency with Semaphores
Task: Throttle 5 concurrent operations so that at most 2 run in parallel.
"""
import asyncio
import time

api_semaphore = asyncio.Semaphore(2)


async def call_restricted_api(client_id):
    async with api_semaphore:
        print(f"Client {client_id} calling limited endpoint...")
        await asyncio.sleep(0.4)
        print(f"Client {client_id} request complete.")


async def main():
    start = time.perf_counter()
    await asyncio.gather(*(call_restricted_api(i) for i in range(5)))
    elapsed = time.perf_counter() - start
    print(f"Throttled execution complete in {elapsed:.2f} seconds.")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Client 0 calling limited endpoint...
Client 1 calling limited endpoint...
Client 0 request complete.
Client 1 request complete.
Client 2 calling limited endpoint...
... (remaining slots)
Throttled execution complete in 1.20 seconds.
```

ACTIVITY 7 - SOLUTION

Exercise 7: Offloading Blocking Synchronous Tasks

How The Solution Works

sync_factorial_calculation is an ordinary, blocking function containing a real time.sleep call, so running it directly inside a coroutine would freeze the entire event loop. Wrapping it in asyncio.to_thread hands it off to a worker thread from asyncio's default thread pool, returning an awaitable immediately. That awaitable is wrapped in create_task alongside async_reporter, so both run concurrently: the reporter keeps ticking on the main thread while the factorial calculation proceeds in the background thread, and the result is collected once both finish.

Key Points

- asyncio.to_thread() runs a blocking function without stalling the event loop.
- The blocking call executes in a separate OS thread, not on the event loop's thread.
- Wrapping the call in create_task lets it run alongside other coroutines rather than blocking main.

Solution Code

```
"""
Solution 7: Offloading Blocking Synchronous Tasks
Task: Execute a blocking CPU calculation safely in an executor thread without stalling loop steps.
"""
import asyncio
import time


def sync_factorial_calculation(number):
    print(f"Factorial calculation started for {number}...")
    total = 1
    for i in range(1, number + 1):
        total *= i
    time.sleep(1.0)  # Blocking sleep simulating heavy processing calculations
    print(f"Factorial calculation for {number} complete.")
    return total


async def async_reporter():
    for i in range(5):
        print(f"Async reporter ticking: {i}...")
        await asyncio.sleep(0.2)


async def main():
    start = time.perf_counter()

    calc_task = asyncio.create_task(asyncio.to_thread(sync_factorial_calculation, 100))
    report_task = asyncio.create_task(async_reporter())

    await report_task
    result = await calc_task

    elapsed = time.perf_counter() - start
    print(f"Calculation Result: {result}")
    print(f"Concurrent workflow complete in {elapsed:.2f} seconds.")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Async reporter ticking: 0...
Factorial calculation started for 100...
Async reporter ticking: 1...
Async reporter ticking: 2...
Async reporter ticking: 3...
Async reporter ticking: 4...
Factorial calculation for 100 complete.
Concurrent workflow complete in 1.00 seconds.
```

ACTIVITY 8 - SOLUTION

Exercise 8: Producer-Consumer Pipeline

How The Solution Works

A bounded asyncio.Queue(maxsize=3) sits between the producer and consumer. The producer coroutine pushes six payloads with await queue.put, which suspends automatically whenever the queue is full, applying backpressure. A single consumer task runs in the background, retrieving items with await queue.get, processing each one under process_semaphore so at most two are handled at a time, and calling queue.task_done() once finished. In main, await pipeline_queue.join() blocks until every put item has been matched by a task_done call, at which point the now-idle consumer task is cancelled.

Key Points

- A bounded Queue naturally paces a fast producer against a slower consumer.
- task_done() combined with queue.join() is how the producer knows all work is finished.
- The consumer's infinite while True loop is safe to cancel once join() returns.
- The Semaphore inside the consumer limits how many items are processed in parallel.

Solution Code

```
"""
Solution 8: Capstone - Producer-Consumer Pipeline with Semaphore Rate-Limiter
Task: Build an end-to-end asynchronous pipeline where producers generate raw API payloads,
an asyncio.Queue pools them, and consumers process them under a Semaphore rate limit.
"""
import asyncio

# Rate limit: max 2 active processing tasks concurrently
process_semaphore = asyncio.Semaphore(2)


async def raw_data_producer(queue, batch_size):
    for i in range(1, batch_size + 1):
        await asyncio.sleep(0.05)  # Simulate payload ingestion
        payload = f"raw_record_{i}"
        print(f"Producer pushed {payload} into queue.")
        await queue.put(payload)
    print("Producer has completed ingestion.")


async def raw_data_consumer(queue):
    while True:
        item = await queue.get()
        async with process_semaphore:
            print(f"Consumer started parsing {item}...")
            await asyncio.sleep(0.2)  # Simulate work
            print(f"Consumer finished parsing {item}.")
        queue.task_done()


async def main():
    pipeline_queue = asyncio.Queue(maxsize=3)

    consumer_task = asyncio.create_task(raw_data_consumer(pipeline_queue))

    await raw_data_producer(pipeline_queue, 6)

    await pipeline_queue.join()
    print("Pipeline work has converged. Shutting down consumer background loop.")
```

```
    if consumer_task:
        consumer_task.cancel()


if __name__ == "__main__":
    asyncio.run(main())
```

Expected Shell Output

```
Producer pushed raw_record_1 into queue.
Producer pushed raw_record_2 into queue.
Consumer started parsing raw_record_1...
Consumer started parsing raw_record_2...
Producer pushed raw_record_3 into queue.
Consumer finished parsing raw_record_1.
Consumer started parsing raw_record_3...
... (converged logs)
Pipeline work has converged. Shutting down consumer background loop.
```

Note: each solution file runs standalone. Timing-sensitive output (elapsed seconds, interleaving of print statements) can vary slightly between runs depending on machine load, but the overall pattern and final results shown above should hold.